

## Trabalho Prático 02 (T02)

### Identificação da Disciplina

| Código da Disciplina | Nome da Disciplina    | Professor              | Período |
|----------------------|-----------------------|------------------------|---------|
| CIC_6MA e EGS_6MA    | Sistemas Operacionais | Jeremias Moreira Gomes | 2026/1  |

### 1. Objetivo Geral

O objetivo deste trabalho é capacitar o aluno a estudar e compreender conceitos envolvendo o desenvolvimento de Sistemas Operacionais. Utilizando o conhecimento teórico adquirido nas aulas e ao longo do curso, o aluno deverá compilar e executar o *bootloader* de um Sistema Operacional simplificado.

Ao cumprir os objetivos do trabalho, o aluno terá adquirido: (i) uma compreensão prática dos conceitos e características-chave sobre a construção e desenvolvimento de Sistemas Operacionais; (ii) maior aprimoramento das suas habilidades de programação, organização de informações e manuseio do ambiente Linux; e (iii) essa experiência proporcionará maior confiança e capacidade para explorar esses conceitos na solução de problemas computacionais.

### 2. Ambiente de Desenvolvimento e Ferramentas

O objetivo deste trabalho é construir entender e desenvolver um programa de inicialização de um Sistema Operacional. Para isso, será necessário algumas ferramentas e conhecimentos que serão explicados e utilizadas ao longo deste documento. As configurações e instalações aqui descritas, foram montadas e testadas no ambiente Linux disponibilizado no laboratório Dell do IDP, o qual possui instalado a ferramenta WSL2, Sistema Operacional Ubuntu 24.04 e o Kernel 6.6.87.2-microsoft-standard-WSL2. Este pode ser replicado para ambientes similares, mas esteja ciente que a montagem em ambientes diferentes podem exigir ajustes nas configurações.

Assim, para a realização deste trabalho, serão necessários as seguintes ferramentas:

- Compilador gcc 13.3.0+
- Emulador QEMU (qemu-system-x86\_64) 8.2.2+
- coreutils 9.4+

### 3. O Computador, O Bootloader e o Sistema Operacional

As ligar um computador, a primeira coisa que acontece é algo chamado de processo de *boot*. Quando a placa-mãe é energizada, ela inicia o *firmware*, incluindo o chipset e outros componentes, para que o processador (CPU) possa começar a trabalhar.

Se a energização ocorrer sem problemas, a CPU começa a operar. Em sistemas com múltiplos núcleos ou processadores, um deles é escolhido como Bootstrap Processor (BSP) para rodar o código inicial da BIOS e do Sistema Operacional. Os outros núcleos permanecem inativos até serem ativados pelo kernel.

Após a inicialização, a CPU entra no **modo real**, uma configuração compatível com o antigo Intel 8086, onde apenas 1 MB de memória está acessível e não há proteção de memória ou privilégios.

### 3.1. Modos de Operação da CPU

Os modos de operação do processador definem como ele gerencia a memória e as tarefas. São três modos principais, resultantes da evolução dos PCs desde o chip Intel 8088.

#### 3.1.1. Modo Real

Esse modo replica o funcionamento do Intel 8088, permitindo acessar até 1 MB de memória, com limitações herdadas do IBM PC original. Ele é usado por sistemas operacionais como o MS-DOS, que funcionam com tarefas únicas. Apesar das restrições, extensores de DOS permitem acesso a memória estendida e processamento de 32 bits, sendo amplamente utilizados por jogos antigos. No Modo Real, o computador inicia e opera com compatibilidade total com processadores antigos.

#### 3.1.2. Modo Protegido

Introduzido com o processador 80286, esse modo elimina o limite de 1 MB, possibilita multitarefa e suporte a memória virtual. Ele protege a memória alocada para cada programa, evitando conflitos e erros. Sistemas como Windows e Linux usam amplamente o Modo Protegido. Processadores 386 e superiores conseguem alternar entre Modo Real e Protegido dinamicamente.

#### 3.1.3. Modo Real Virtual

Este é um recurso do Modo Protegido que simula o Modo Real, permitindo que programas DOS rodem em sistemas operacionais modernos. Ele cria "máquinas virtuais" com até 1 MB de espaço de memória, úteis para rodar aplicativos legados em ambientes multitarefa, como o Windows 95. Introduzido com o processador 386, ele mantém compatibilidade com softwares mais antigos.

Cada um desses modos reflete a evolução dos processadores que atendiam e atendem a diferentes necessidades. O **Modo Real** mantém compatibilidade com sistemas legados, sendo usado em aplicações específicas, como inicialização do hardware. O **Modo Protegido** é a base dos sistemas modernos, oferecendo multitarefa, proteção de memória e suporte a memória virtual.

---

Voltando à CPU, que inicia sua execução no modo real, ela começa a executar instruções a partir do endereço `0xFFFFFFFF0`, que é chamado de **vetor de reset**, onde a CPU buscará a primeira instrução a ser executada após um reset. Este endereço é mapeado para o início do código da BIOS (Basic Input/Output System)<sup>1</sup>.

Neste ponto, a memória RAM ainda não está pronta para uso, então a BIOS é executada diretamente a partir da ROM (Read-Only Memory). A BIOS inicializa o POST (Power-On Self Test), que verifica o hardware do computador e também configura os dispositivos PCI e cria tabelas que descrevem os dispositivos do sistema, seguindo o padrão ACPI (Advanced Configuration and Power Interface).

---

<sup>1</sup> Aqui talvez possa ocorrer um problema de terminologia. BIOS, hoje em dia, é um termo mais informal que refere-se ao código que inicia o computador, cujo a tecnologia mais atual é o UEFI. Tanto o UEFI quanto a BIOS acabam sendo referenciadas como BIOS.

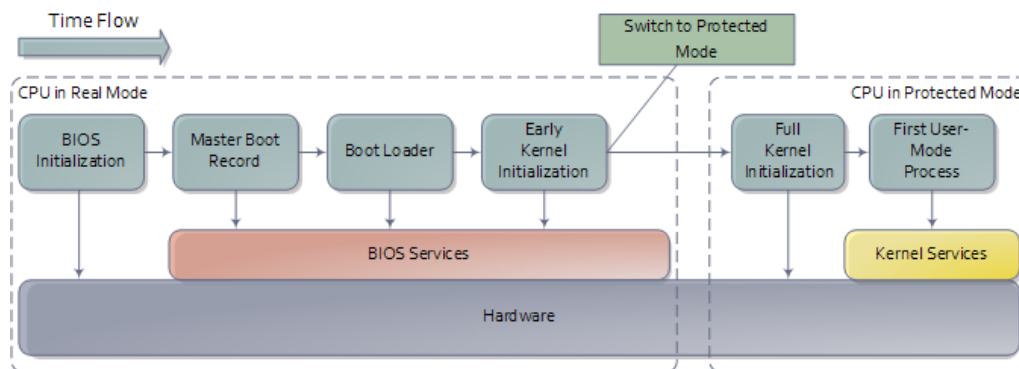
Somente após a conclusão do POST, a BIOS começa o processo de inicialização do sistema operacional, que é carregado a partir de um dispositivo de armazenamento, como um HD, SSD, CD, DVD, USB, etc. A BIOS procura por um dispositivo inicializável, que possui dados específicos utilizados para carregar o sistema operacional, chamado de *bootloader*.

Em um dispositivo denominado inicializável (que possui um setor de boot), a BIOS lê o primeiro setor do disco (primeiros 512 bytes), chamado de Master Boot Record (MBR), que contém dois componentes:

- Um programa responsável por carregar o sistema operacional; e
- Uma tabela de partições que descreve como o disco está dividido.

A BIOS não interpreta esse conteúdo. Ela carrega a MBR na memória (no endereço 0x7c00) e transfere a execução para lá, permitindo que o código do MBR assuma o controle.

O código na MBR é normalmente o responsável por carregar o sistema operacional. Este pode ser um Windows, Linux, um gerenciador de sistemas como GRUB ou LILO, ou até mesmo um programa personalizado como um vírus. A partir desse ponto, o sistema (operacional) começa a ser carregado, dando continuidade ao processo de inicialização.



### 3.2. Bootloader

O bootloader é um programa carregado na memória RAM do computador pela BIOS, após a conclusão do POST. Ele tem como principal função auxiliar o computador a localizar o sistema operacional a ser carregado, que funciona da seguinte forma:

- Quando a BIOS precisa carregar um sistema operacional, ela percorre os dispositivos disponíveis, como HDDs, CD-ROMs, USBs ou disquetes, verificando se algum é inicializável e contém um bootloader;
- Esse processo inclui:
  - Ler os primeiros 512 bytes (setor de boot) do dispositivo e armazená-los na posição de memória 0x7c00;
  - Verificar se os dois últimos bytes são 0xaa55, um número mágico que indica à BIOS que o disco é inicializável e contém um bootloader; e
- Transferência de Controle: Após localizar o bootloader, a BIOS transfere a execução para o endereço 0x7c00, onde o código do bootloader é executado.

## 4. Atividade

Neste trabalho, você deverá criar um bootloader simples que carregue um kernel simplificado, disponibilizado especificamente para esta atividade. Para isso, as seções a seguir irão detalhar o funcionamento e a criação de um bootloader de exemplo, bem como os detalhes necessários para a conclusão da atividade.

### 4.1. Hello World Bootloader

Obrigatoriamente, o bootloader deve ter 512 bytes de tamanho, onde os dois últimos bytes devem ser 0xaa55. Essa é a forma que a BIOS identifica um dispositivo como inicializável. A figura 1 mostra o formato do bootloader.

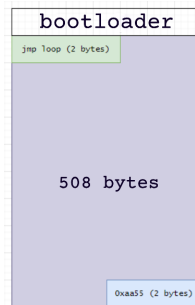


Figura 1. Formato do Bootloader

Na figura, o bootloader é composto por apenas um salto que aponta para o início do próprio código. Apesar de a instrução ocupar apenas 2 bytes, ainda é necessário o preâmbulo do código com informações que ditam o modo de operação, que deve ser o modo real, ou seja, o processador deve estar em modo real quando o bootloader estiver sendo executado e todas as instruções devem ser compatíveis com o processador Intel 8086 (16 bits). Além disso, é necessário o preenchimento do espaço não utilizado com zeros, até o local da assinatura do bootloader. A instrução para o modo de operação é a `code16`. A parte do código que mostra o funcionamento similar à figura apresentada é a seguinte:

```
.code16                                # Indica o modo real
.intel_syntax noprefix
.global _start

_start:
    jmp _start                        # Salta para o início do código

end:
    # Preenchimento com zeros
    # (insira o código aqui)

    .byte 0x55, 0xaa                # Assinatura do bootloader
```

Para tornar o programa um bootloader funcional, falta apenas o preenchimento com zeros, que pode ser feito de diversas formas diferentes. Uma delas é utilizando a diretiva `.fill` do assembler do gcc. Essa diretiva funciona da seguinte forma:

```
.fill <quantidade>, <tamanho>, <valor>
```

Onde <quantidade> é a quantidade de vezes que o valor será repetido, <tamanho> é o tamanho do valor (1 byte, 2 bytes, 4 bytes, etc) e <valor> é o valor que será repetido. Além disso, o gcc também possui uma outra diretiva que pode auxiliar na criação do bootloader, que é o ponto (.), que indica o local atual do código. Combinando o uso dessas diretivas, complete o código acima para que o bootloader tenha exatamente 512 bytes de tamanho, finalizando com a assinatura 0x55, 0xaa.

Para montar o código, utilize o seguinte comando:

```
$ as -o bootloader.o bootloader.s
$ ld -o bootloader --oformat binary -Ttext 0x7c00 bootloader.o
```

O parâmetro `-Ttext 0x7c00` indica ao `ld` que o código deve ser carregado a partir do endereço 0x7c00, que é o endereço que a BIOS irá colocar o bootloader na memória. Já o parâmetro `--oformat binary` indica que o arquivo de saída deve ser um arquivo binário, que é o formato esperado pela BIOS (se você olhar o arquivo gerado com o comando `hexdump`, perceberá que ele é similar às atividades de programação em assembly, anteriormente executadas, com a diferença de ser especificamente para 16 bits). Tendo o arquivo `bootloader` gerado, você pode utilizar o QEMU para testar o bootloader. Para isso, utilize o seguinte comando:

```
$ qemu-system-x86_64 bootloader
```

Se não houver erros, você verá uma tela preta com um cursor piscando, após a mensagem "Booting from Hard Disk...", que indica o funcionamento do laço infinito do bootloader.

## 4.2. Interrupções

Agora que você tem uma versão simplificada de um bootloader, já é possível utilizar funcionalidades providas por meio de interrupções.

As interrupções são eventos que ocorrem durante a execução de um programa, que podem ser causadas por hardware ou software. As interrupções permitem que o sistema (no nosso caso, o bootloader) responda a eventos externos, como pressionar uma tecla, ou a eventos internos, como a conclusão de uma operação de I/O.

Uma lista de interrupções muito famosa<sup>2</sup> é a Ralph Brown's Interrupt List (<https://www.ctyme.com/intr/int.htm>) que contém uma lista de interrupções do DOS, BIOS, e outras informações úteis para programadores, de uma maneira mais acessível, que foi construída de uma maneira colaborativa, em uma época mais antiga onde encontrar informações sobre interrupções era um processo mais complicado.

Por exemplo, para interagir com o vídeo, você pode utilizar a interrupção 0x10 (<https://www.ctyme.com/intr/int-10.htm>), que permite escrever caracteres na tela. Para utilizar o modo texto, deve-se atribuir os seguintes valores aos registradores:

- AH = 0x0e      (função de escrita de caractere no modo teletexto (clique aqui para detalhes))
- AL = <valor>    (caractere a ser impresso)

<sup>2</sup>tem até página na wikipedia

- BH = 0x00 (página de vídeo)

Assim, para imprimir o caracter A na tela, você pode fazer o seguinte:

```
mov al, 0x41
mov ah, 0x0E
mov bh, 0x00
int 0x10
hlt
```

E isso irá imprimir o caracter A na tela. A instrução `hlt` (*halt*), após a interrupção, faz com que o processador pare de executar instruções. Teste esta funcionalidade no seu bootloader.

### 4.3. Carregando o Kernel para Memória a partir do Bootloader

Agora que você já entende o mecanismo de interrupções em conjunto com o bootloader, o trabalho de fato pode ser concluído. Para isso, você deverá executar, no seu bootloader, as seguintes tarefas:

1. Desabilitar as interrupções;
2. Limpar os registradores de segmento;
3. Mover o ponteiro de pilha para o endereço 0x7c00;
4. Habilitar as interrupções;
5. Resetar o sistema de disco (interrupção 0x13);
6. Setar o segmento estendido para 0x7E0;
7. Carregar o kernel para a memória;
8. Inserir o Verificador da Matrícula (VM) no registrador AX;
9. Saltar para o kernel (0x7E00);
10. Criação do disco e cópia do bootloader e kernel para o disco; e
11. Executar o disco a partir do QEMU.

#### 4.3.1. Desabilitar as Interrupções

Para desabilitar as interrupções, você pode utilizar a instrução `cld`.

#### 4.3.2. Limpar os Registradores de Segmento

Você deverá inserir o valor 0x0 nos registradores DS, ES e SS. A partir disso, em passos futuros os endereços do segmento estendido serão utilizados apropriadamente.

#### 4.3.3. Mover o Ponteiro de Pilha para o Endereço 0x7c00

Para mover o ponteiro de pilha para o endereço 0x7c00, você deverá inserir o valor 0x7c00 no registrador SP.

#### 4.3.4. Habilitar as Interrupções

Para habilitar as interrupções, você pode utilizar a instrução `sti`.

#### 4.3.5. Resetar o Sistema de Disco

A interrupção 0x13 é utilizada para operações de disco. Para resetar o sistema de disco, a função é a 0x0, que deverá ser o valor do registrador AX.

Consulte este link, para mais informações sobre a interrupção 0x13.

#### 4.3.6. Setar o Segmento Estendido

Para setar o segmento estendido para 0x7E0 (onde o kernel será carregado), você deverá inserir o valor 0x7E0 no registrador ES. Repare que o valor é 0x7E0 e não 0x7E00, pois no modo real, os endereços que utilizam o segmento estendido são acessados a partir do seguinte:

```
endereço = (segmento << 4) + offset
```

Para mais detalhes sobre o funcionamento de segmentos no modo real, você pode ler o artigo deste link.

#### 4.3.7. Carregar o Kernel para a Memória

Para carregar o kernel para a memória, você deverá utilizar a interrupção 0x13, função 2 (Read Disk Sectors (link para detalhes)). Um disco possui cilindros, cabeças e setores, que são utilizados para acessar os dados. A figura 2 mostra a organização de um disco.

As informações nos registradores, para a operação de leitura de disco para a memória, são as seguintes:

- AH = 0x02 (função de leitura de disco)
- AL = 0x04 (número de setores a serem lidos (o kernel possui entre 1500 e 2000 bytes))
- CH = 0x00 (cilindro)
- CL = 0x02 (setor)
- DH = 0x00 (cabeça)

Após “setar” os valores nos registradores, você deverá chamar a interrupção 0x13, que irá carregar o conteúdo apontado para o segmento estendido (0x7E0).

#### 4.3.8. Inserir o Valor Verificador da Matrícula no Registrador AX

Você deverá mover o valor referente ao verificador da sua matrícula ( $VM$ ) para o registrador AX. O verificador é calculado da seguinte forma:

$$VM = \sum_{i=1}^n (d_i * i)^3 \% 4093 \quad (1)$$

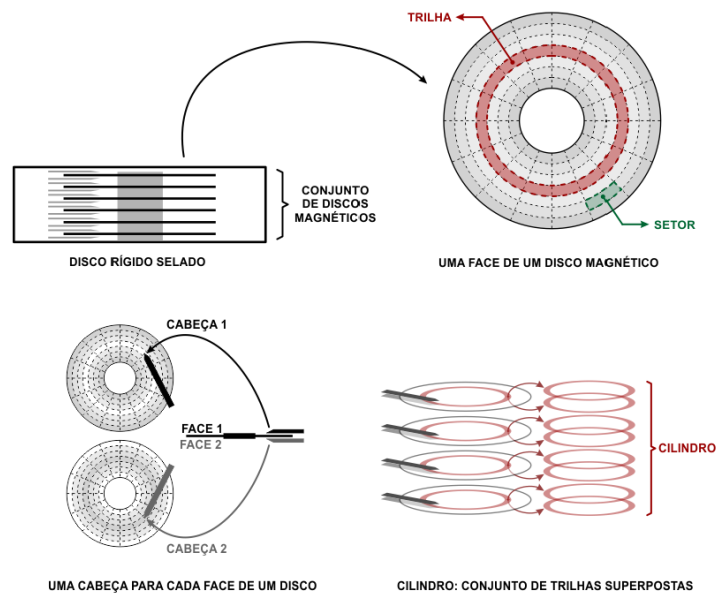


Figura 2. Diagrama de um Disco Rígido

onde  $d_i$  é o dígito  $i$  da matrícula, da esquerda para a direita, e  $n$  é o número de dígitos da matrícula.

#### 4.3.9. Saltar para o Kernel

Agora que o kernel está carregado na memória, você deverá saltar para o início do mesmo que é o endereço `0x7E00` com a instrução `jmp`. Essa é a última instrução do seu bootloader, fora o complemento com zeros e a assinatura do mesmo.

#### 4.3.10. Criação do Disco e Cópia do Bootloader e Kernel para o Disco

Para executar o bootloader e o kernel, ambos os programas são copiados para um disco. Para isso, será utilizado o utilitário `dd`, que é uma ferramenta de linha de comando para realizar operações de cópia de bytes.

Para criar um disco chamado `disco.img`, faça o seguinte:

```
$ dd if=/dev/zero of=disco.img bs=1024 count=720
```

onde `if` é o fluxo de entrada, `of` é o fluxo de saída, `bs` é o tamanho do bloco e `count` é a quantidade de blocos. **Atenção:** tome cuidado inclusive com os espaçamentos ao executar o comando, pois ele irá sobrescrever qualquer caminho apontado no parâmetro `of`.

A execução correta do comando irá criar um arquivo de 720KB chamado `disco.img`, totalmente preenchido com zeros. Para copiar o bootloader para o início do disco, você pode fazer o seguinte:

```
$ dd if=bootloader of=disco.img conv=notrunc seek=0
```



onde `conv=notrunc` indica que o arquivo de saída não deve ser truncado, e `seek=0` indica que a cópia deve começar no início do disco.

Agora que o bootloader está no disco, falta copiar o conteúdo arquivo `kernel` para o segundo setor do disco. Para isso, você pode fazer o seguinte:

```
$ dd if=kernel of=disco.img conv=notrunc bs=512 seek=1
```

Com isso, o disco está pronto para ser utilizado.

#### 4.3.11. Executar o Disco a partir do QEMU

Para executar o disco a partir do QEMU, você pode fazer o seguinte:

```
$ qemu-system-x86_64 disco.img
```

Que, caso seja executado corretamente, irá mostrar a resposta para a atividade (também dever-se-á submeter o código gerado para o bootloader).

### 5. Debugging e Testes (seção extra)

É possível utilizar o GDB para depurar o bootloader. Para isso, você pode executar o QEMU com a opção `-s` para habilitar o servidor GDB, que irá executar na porta 1234, e `-S` para pausar a execução do QEMU até que o GDB se conecte. O comando é o seguinte:

```
$ qemu-system-x86_64 -s -S disco.img
```

Após isso, abra o GDB no terminal e conecte-se ao QEMU em execução, digitando o seguinte:

```
$ gdb
(gdb) target remote :1234
```

E isso irá conectar o GDB ao QEMU, permitindo que você depure o bootloader.

#### 5.1. Dicas de Depuração

Para parar a execução no início do bootloader, insira um `breakpoint` no endereço `0x7c00`:

```
(gdb) break *0x7c00
```

E pressione `continue` para continuar a execução até o ponto de parada.

Como o modo de execução é o modo real, algumas instruções podem não aparecer legíveis no GDB (então não se assuste). Além disso, a chamada a interrupções, pode realizar operações que não são visíveis no GDB, que acabam por bagunçar as instruções atuais visualizadas. Para evitar isso, sempre que uma interrupção estiver para ser executada, insira um `breakpoint` no endereço após a interrupção, e utilize um `continue` para que o GDB passe por ela e pare apenas após a execução da interrupção.

## 5.2. Sumário

Ao final do trabalho, o aluno/grupo deverá submeter a resposta coletada a partir da execução correta do disco, e submeter o código do bootloader gerado.

## 5.3. Restrições deste Trabalho

Esta atividade deverá ser realizada de maneira **individual**.

## 5.4. Datas Importantes

Esse trabalho poderá ser submetido até o dia 07/06/2026 (domingo) - 23:55h.

## 5.5. Por onde será a entrega o Trabalho?

O trabalho deverá ser entregue via Moodle na disciplina de Sistemas Operacionais em uma atividade chamada “Trabalho Prático 02”, para submissão da resposta.

## 5.6. O que deverá ser entregue, referente ao trabalho?

Deverá ser entregue a resposta final coletada a partir da conclusão da atividade e o código do bootloader gerado.

## 5.7. Observações importantes

- **Não deixe para fazer o trabalho de última hora.**
- Caso não faça o trabalho no laboratório da disciplina (Dell), fique atento para a versão de todos os softwares utilizados.
- **Não deixe para fazer o trabalho de última hora.**

## 6. Bibliografia

1. Bootloader, <https://wiki.osdev.org/Bootloader>, acesso em 2026.
2. Modos do Processador, <https://flint.cs.yale.edu/feng/cos/resources/BIOS/procModes.htm>, acesso em 2026.
3. What is UEFI, and How Is It Different from BIOS?, <https://www.howtogeek.com/56958/htg-explains-how-uefi-will-replace-the-bios/>, acesso em 2026.